Anthony Pham
Dr. Eman Ramadan
CSCI 4211
14 October 2025

Project 1 Report

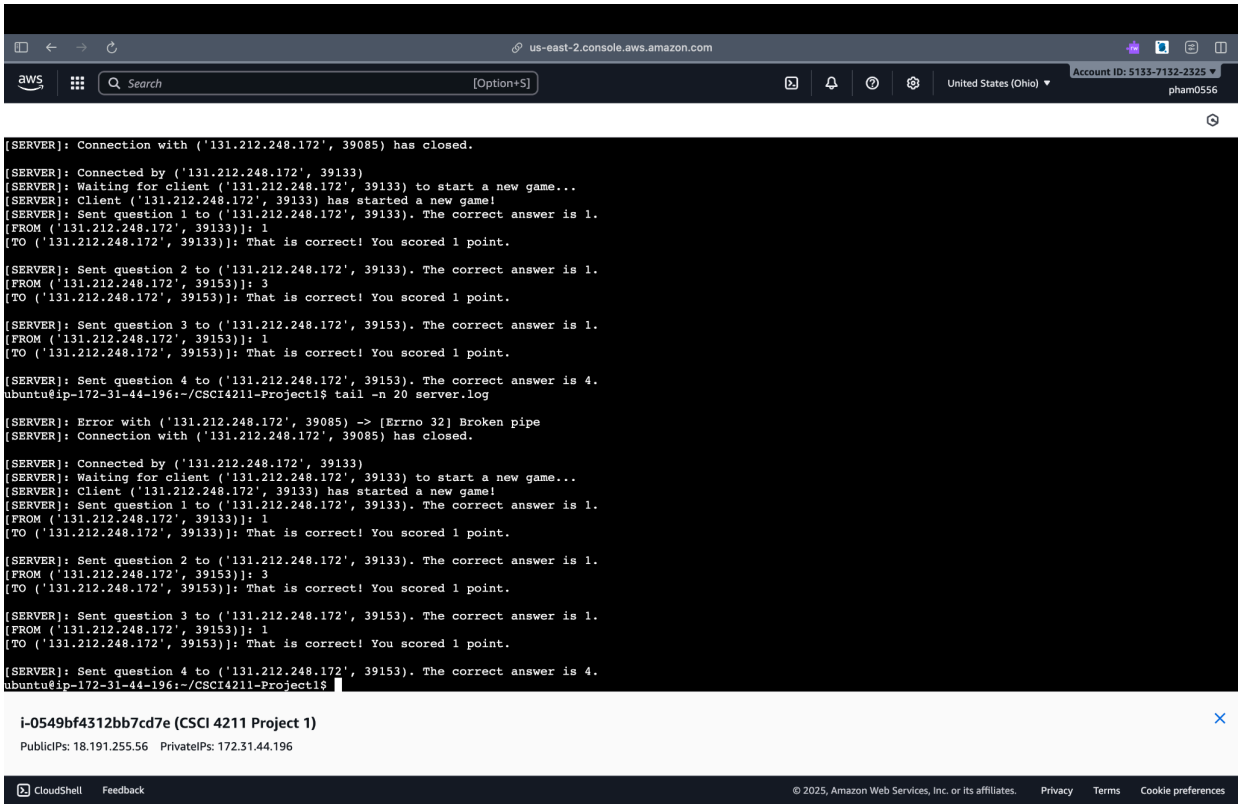
Implementing Multiple Client Functionality

To have the server be able to handle multiple clients simultaneously, I opted to use the *threading* library. This library provides the *Thread* class, which allowed me to assign each client socket to an independent thread running the trivia_game() function. Having multiple threads running concurrently meant that I needed to be mindful of how each thread accesses the shared resources on my laptop or server. Most importantly, I needed to ensure that only one thread accesses the client_socket set at a time when adding or removing from the set. This is where the Lock class comes in. When a connection_socket (or client_socket) is added, discarded, shut down, or closed, a Lock object is there to make sure that only one thread is accessing the set at a time.

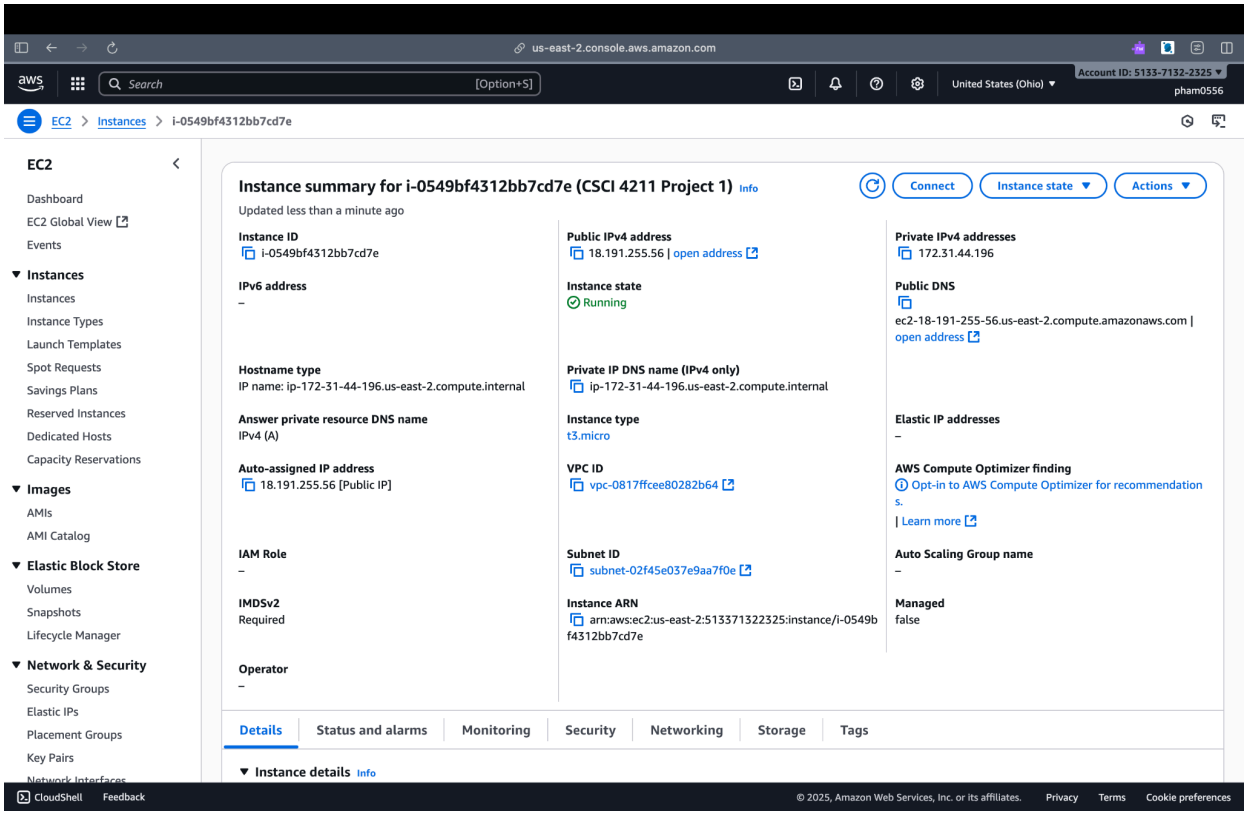
An additional functionality I implemented was a timeout for each client, where the server will close the connection socket if nothing is heard from the client for a minute. This is to prevent the set of client sockets from getting cluttered with useless connections.

Deploying the Server

I was successful in deploying the server in the cloud through AWS's EC2 cloud service. The server has a public IP address of 18.191.255.56 and uses port 5001.



Terminal output is only printed in the server's deployed code for debug purposes.



Cloud Setup Experience

Navigating AWS and the initial server setup was the biggest challenge for me, as I had never deployed a server nor used AWS. AWS offers a *large* number of services, so it was difficult and daunting to decide which service to use to deploy my code. After some research, I decided to go with EC2 as it has plenty of online resources, is accessible to me as a free user, and seemed to fit my needs the most without any extra fluff while also giving me control over configuring the server.

I found it a bit straightforward to create the instance, and it helped that the AWS website provides you with a web terminal to connect to the server. I was then given a public IPv4 address to plug into the client code and set the server up to use port 5001 for connecting. Then I uploaded my code to a GitHub repo from my laptop, where I cloned the code from the server and ran the Python server code. But that's where I ran into my second issue: after a while, the SSH connection to the server would timeout and disconnect, which meant that the server code would stop running.

Earlier, I had assumed that the server setup was complete, but when I attempted to connect to the server again after a couple of hours, the client was unable to connect to the server. That's when I learned that the SSH session isn't persistent. The solution was to use the *nohup* command, which ignores the hangup signal when the SSH disconnects and keeps the server code running in the background. I had the server output go to a *server.log* file so that I can access the output at a later time. After trying this solution, the server seems to be persistently running when I tried connecting to the server at several later attempts.